

PROGRAMME DE FORMATION · VERSION 3 · 2026-09-13

Actualisation de Java 11 à Java 17

Préparez une migration de Java 11 vers Java 17 et modernisez le code là où cela améliore sa lisibilité. Le fil rouge couvre compatibilité, nouvelles constructions du langage, tests et procédure de basculement.

3 jours · 21 heures

Intermédiaire · Hybride

1 272 € HT

Tarif indicatif, dates, disponibilité et modalités contractuelles à confirmer avec le centre.

À qui s'adresse cette formation ?

Développeurs Java et responsables techniques préparant une migration d'un projet Java 11 vers Java 17.

Prérequis

Cette formation présente les nouveautés de Java 11 et Java 17 pour un profil maîtrisant déjà le langage Java. Pour suivre cette formation, il est donc indispensable de bien connaître la version Java 8.

Objectifs pédagogiques

- Connaître les nouveautés apportées par les versions de Java 9 et ultérieures.
- Connaître, version après version, les nouveautés apportées aux dernières versions LTS de Java
- Mettre en pratique les nouveautés significatives et les plus importantes

Le cas fil rouge

Migrer une application Java 11 vers Java 17 en distinguant nouveautés disponibles, fonctionnalités preview et problèmes de compatibilité.

Préparation

Java 11, compilation et tests ; JDK 11 et 17 isolés, Maven ou Gradle, application de migration préparée.

Progression détaillée

Répartition pédagogique proposée sur une base de 7 heures par jour, pauses exclues. À ajuster avec le formateur au positionnement initial. Les approfondissements « Pour aller plus loin » sont sélectionnés par le formateur selon le positionnement et le temps disponible ; ils ne constituent pas des heures supplémentaires garanties.

Jour 1 · 7 heures

1. Établir le diagnostic de migration

210 minutes · apports, exercice et retour

- Inventorier dépendances, plugins et options JVM.
- Situer les changements de Java 9 à 11 utiles au contexte.
- Identifier usages internes et composants à mettre à jour.

- Analyser les dépendances avec les outils du JDK, repérer modules et accès internes, puis distinguer incompatibilité de compilation et d'exécution.

NOTIONS ET MÉTHODES EN DÉTAIL

- Inventorier JDK, dépendances, plugins de build et options JVM du projet de référence.
- Séparer compatibilité source, binaire et comportement à l'exécution.
- Repérer utilisation d'API internes et accès réflexifs affectés par l'encapsulation renforcée.

Atelier prévu

Construire une matrice de compatibilité du projet et obtenir une exécution de référence sous Java 11.

Livrable attendu

Matrice de compatibilité et risques de migration.

Vérification en atelier

Le diagnostic produit une liste d'écarts reproductibles et non une simple montée de version.

Pour aller plus loin · selon positionnement et temps disponible

Identifier une bibliothèque incompatible avec l'encapsulation renforcée.

2. Moderniser les expressions et le texte

210 minutes · apports, exercice et retour

- Employer expressions switch et vérifier l'exhaustivité.
- Utiliser blocs de texte avec maîtrise des espaces et retours.
- Revoir APIs de chaînes et pratiques de code compatibles Java 17.
- Employer expressions switch et blocs de texte en préservant lisibilité et comportement.

NOTIONS ET MÉTHODES EN DÉTAIL

- Utiliser expressions switch avec valeurs de retour et branches exhaustives adaptées au cas.
- Employer text blocks en vérifiant indentation et contenu réel de la chaîne.
- Comparer avant/après sur lisibilité et couverture des branches.

Atelier prévu

Refactorer un parseur et ses chaînes multilignes en comparant les sorties avant et après modification.

Livrable attendu

Exemples modernisés et tests de non-régression.

Vérification en atelier

La modernisation conserve les valeurs exactes et tous les cas métier testés.

Pour aller plus loin · selon positionnement et temps disponible

Réécrire un traitement avec tests sur valeurs nulles et cas limites.

Jour 2 · 7 heures

3. Modéliser avec records et types scellés

210 minutes · apports, exercice et retour

- Comprendre composants, égalité et immutabilité limitée des records.

- Encadrer une hiérarchie avec sealed et permits.
- Distinguer nouveauté stable et fonctionnalité encore en preview dans Java 17.
- Utiliser records pour données et types scellés pour hiérarchies contrôlées ; expliciter leurs limites.

NOTIONS ET MÉTHODES EN DÉTAIL

- Définir un record pour une donnée immutable dans sa structure et examiner la mutabilité de ses composants.
- Construire une hiérarchie sealed avec sous-types autorisés et responsabilités cohérentes.
- Distinguer objet de données et entité à identité ou cycle de vie mutable.

Atelier prévu

Remplacer un DTO par un record et modéliser un résultat de traitement avec une hiérarchie fermée.

Livrable attendu

Modèle de types et invariants vérifiés.

Vérification en atelier

Le choix du record ou du type scellé répond au modèle du domaine.

Pour aller plus loin · selon positionnement et temps disponible

Comparer record immuable en surface et objet contenant une collection mutable.

4. Adapter les usages du langage et des bibliothèques

210 minutes · apports, exercice et retour

- Utiliser le pattern matching pour instanceof.
- Examiner APIs supprimées, dépréciées et encapsulation renforcée.
- Ajuster réflexion, bibliothèques et configuration modulaire si nécessaire.
- Distinguer fonctionnalités finalisées et options preview ; vérifier les changements d'API utilisés.

NOTIONS ET MÉTHODES EN DÉTAIL

- Utiliser pattern matching pour instanceof sans mélanger fonctionnalités de préversion et fonctionnalités finalisées.
- Vérifier les changements de bibliothèques réellement utilisés par le projet.
- Retirer contournements devenus inutiles après validation de la compatibilité.

Atelier prévu

Diagnostiquer un accès réflexif cassé et choisir une correction de dépendance plutôt qu'une option de contournement permanente.

Livrable attendu

Liste des adaptations et décisions de compatibilité.

Vérification en atelier

Le code cible compile avec la version annoncée sans activer de préversion non prévue.

Pour aller plus loin · selon positionnement et temps disponible

Remplacer un accès réflexif fragile par une API prise en charge.

Jour 3 · 7 heures

5. Valider runtime et performances

210 minutes · apports, exercice et retour

- Rejouer tests unitaires et intégration sous Java 17.
- Mesurer démarrage, mémoire et temps de traitement sur charge identique.
- Examiner logs et options de collecte mémoire effectivement utilisées.
- Mesurer démarrage, allocation, GC et débit avec charge représentative et warm-up.

NOTIONS ET MÉTHODES EN DÉTAIL

- Comparer démarrage, mémoire et latence avec charge et configuration documentées.
- Examiner logs JVM et comportement du ramasse-miettes avant de modifier les options.
- Exécuter tests unitaires, intégration et parcours sensibles sur les deux environnements.

Atelier prévu

Comparer les deux runtimes avec protocole stable et analyser une régression observée sur l'application de démonstration.

Livrable attendu

Rapport de performance reproductible.

Vérification en atelier

Un écart de mesure est analysé avant d'être attribué à la nouvelle version du JDK.

Pour aller plus loin · selon positionnement et temps disponible

Expliquer une variation de latence sans l'attribuer automatiquement à la version Java.

6. Préparer le basculement

210 minutes · apports, exercice et retour

- Construire un artefact reproductible avec le JDK cible.
- Mettre à jour CI, images et documentation de support.
- Prévoir validation, surveillance et retour à la version précédente.
- Prévoir CI sur Java 17, packaging, déploiement progressif et procédure de retour arrière.

NOTIONS ET MÉTHODES EN DÉTAIL

- Préparer image d'exécution, chaîne CI et documentation des versions supportées.
- Définir critères de bascule et procédure de retour à l'environnement précédent.
- Recetter erreurs, encodage et intégrations externes à partir du dossier de migration.

Atelier prévu

Présenter un dossier de migration avec modifications, tests, dépendances et procédure de retour arrière.

Livrable attendu

Plan de bascule et critères de validation.

Vérification en atelier

Le dossier comporte une preuve de build, de recette et de retour arrière.

Pour aller plus loin · selon positionnement et temps disponible

Simuler un échec de production malgré des tests unitaires réussis.

Évaluation finale

Mise en situation individuelle et restitution commentée, incluses dans la dernière séquence. Durée indicative : 60 minutes.

- Les fonctionnalités utilisées sont disponibles dans Java 17 avec leur statut exact.
- L'application conserve son comportement testé.
- Les différences de performance sont mesurées et contextualisées.

Méthodes et ressources

Apports illustrés, exercices progressifs, ateliers sur un cas fil rouge et retour argumenté sur les productions. Les ateliers et l'évaluation font partie des heures indiquées.

Les ateliers décrivent les travaux prévus. Leurs fichiers, corrigés et évaluations en ligne sont à préparer, sauf ressources explicitement disponibles dans le module pilote.

Accès, accompagnement et contact

Pour obtenir plus de détails ou le programme détaillé validé, contactez le centre : 09 54 70 23 80 — contact@beyondexpertise.eu.

Positionnement et vérification des prérequis avant inscription. Délais, calendrier et aménagements à convenir avec le centre. Pour un besoin d'accessibilité, décrivez le besoin pratique sans diagnostic médical.

Programme enrichi proposé, à valider par le formateur avant animation. Les approfondissements sont adaptés au niveau et au temps disponible. Aucune certification professionnelle ni prise en charge CPF ou OPCO n'est garantie. Démarche Qualiopi en cours ; certification non acquise.

Références pour approfondir

- [Oracle](#) · [changements du JDK 17](#)